

Diffusion-MoE Architecture Reference

The diffusion-map-routed mixture-of-experts design, end to end

Living reference · [diffusion-moe project](#) · companion to [methodology.md](#) and [phase1_findings_report.md](#)

1. The core idea, in one paragraph

`DiffusionMoETransformer` is a standard pre-norm decoder-only transformer (RoPE attention, RMSNorm, SwiGLU) in which some or all layers' single dense FFN is replaced by a **mixture of narrower FFN experts**, and — the project's distinguishing bet — **which expert a token goes to is decided by that token's coordinates in a diffusion-map embedding of the layer's own post-attention activations**, not by a directly-learned linear router (Switch Transformer's approach) or by raw-residual-stream cosine similarity. The diffusion map is refit periodically from the batch itself (an unsupervised, non-parametric embedding step), while the expert centroids living in that embedding and the expert FFNs themselves are ordinary trained parameters. Sections 2–6 below trace this pipeline component by component, grounded directly in the current code (`src/diffusion_moe/`), not a design intent — every claim here should hold under `grep` .

2. Full model: `DiffusionMoETransformer` (`models/moe_model.py`)

```
input_ids (batch, seq)
|
▼
token_embedding: Embedding(vocab_size, d_model)
|
▼
blocks[0] — blocks[1] — ... — blocks[n_layers-1]      (n_layers stacked, pre-norm)
|               each block is EITHER:
|               • TransformerBlock (dense — plain SwiGLU FFN), or
|               • one of 4 MoE layer variants (§5), per `layers_to_replace`
|
▼
final_norm: RMSNorm(d_model)
|
▼
lm_head: Linear(d_model, vocab_size)  [tied to token_embedding.weight by default]
|
▼
logits (batch, seq, vocab_size)
```

- **layers_to_replace** (a `set[int]` , default = every layer index `0..n_layers-1`): the only structural knob between "fully dense baseline" (`layers_to_replace=[]`), "fully diffusion-MoE" (the default), and the selective architecture Phase 1's oracle-ceiling result (`phase1_findings_report.md` §3.3) argues for — e.g. `[2, 4]` only, once wider evidence (the in-progress 32-layer sweep) confirms which layers actually warrant it.
- **router** (str, default `"diffusion"`): selects which of the 4 MoE layer classes (§5) fills every replaced slot — `"diffusion"` | `"cosine"` | `"switch"` | `"random"` . Mixing router types across different layers of the same model isn't supported; it's one global choice per model instance.

- **Forward pass returns three things**, not just logits: (logits, activations, router_outputs) . activations[i] is every layer's residual-stream output (dense or MoE alike). router_outputs[i] is present only for MoE layers — each variant's own aux dict, keyed by layer index — and is what training/losses.py 's total_loss consumes for the load-balance/separation penalties (§6).

3. The dense block (models/transformer_block.py)

Every non-replaced layer, and the implicit reference architecture every MoE variant's attention/norm stack copies:

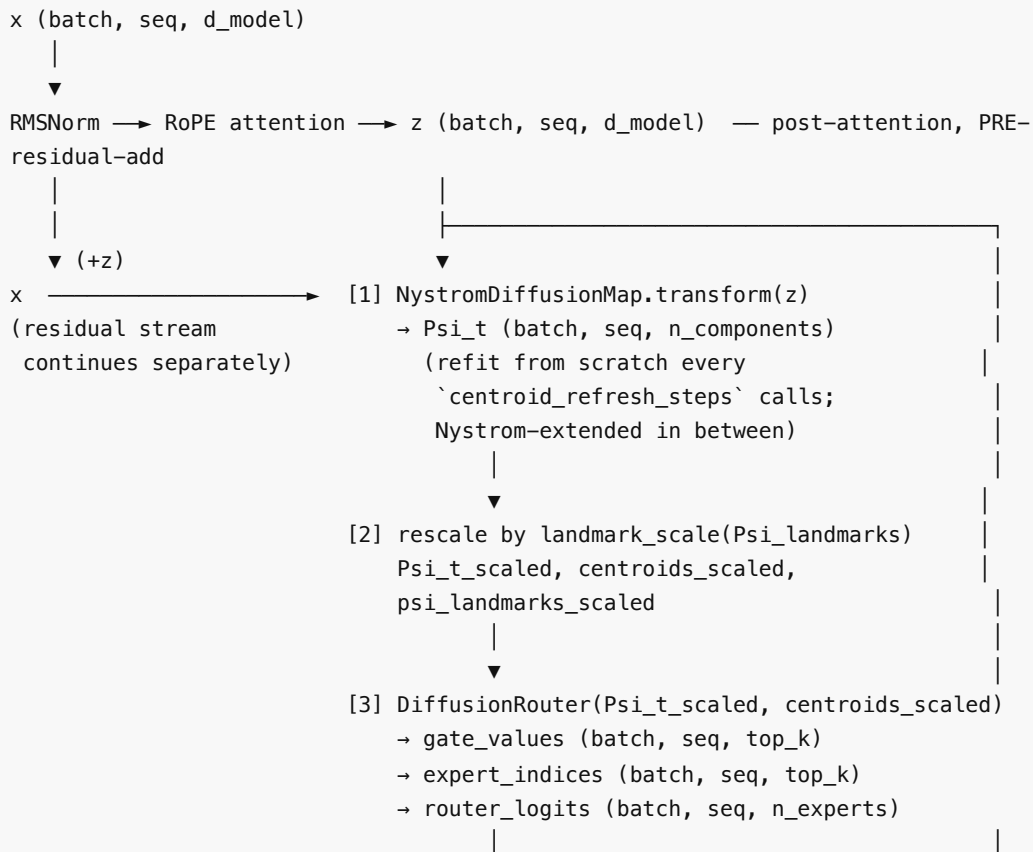


SwiGLU FFN: `down_proj(silu(gate_proj(x)) * up_proj(x))` , `hidden_dim = ffn_dim` (the full dense width — `models/ffn.py` 's `FeedForward` , distinct from `ExpertFFN` 's narrowed version below).

4. The diffusion-routed MoE block

(models/moe_layer.py::DiffusionMoELayer)

This is the architecture's central novelty. Same attention/norm skeleton as §3, but the FFN sub-block is replaced by a 5-stage routing-and-dispatch pipeline:



```

      ↓
[4] dispatch_and_aggregate(ffn_norm(x), ...) ←
    → expert_out (batch, seq, d_model)
      ↓
      ↓
[5] output = x + expert_out    (residual add)

```

Per-stage detail:

1. **Diffusion embedding** (`geometry/nystrom.py::NystromDiffusionMap`) — fit on `min(n_landmarks, n_tokens)` k-means++ landmarks selected from the batch's own post-attention activations (always upcast to float64, on CPU, via scikit-learn regardless of the model's own dtype/device); a Gaussian kernel with median-heuristic bandwidth is Coifman–Lafon normalized (`alpha` -weighted degree normalization) into a Markov transition matrix, eigendecomposed, and the **trivial top eigenpair (eigenvalue ≈ 1) is discarded** so index 0 of every returned array is the first informative diffusion direction. Diffusion coordinates are `Psi_t = eigenvector · eigenvalue^t` — an explicit low-pass filter on the kernel's spectrum, sharper for larger `t`. Landmarks and the eigensystem are **refit only every `centroid_refresh_steps` forward calls** (expensive step); every other call reuses the frozen landmarks via the Nystrom out-of-sample extension formula (`transform`), which is why the class exists at all — an exact diffusion map is prohibitive to recompute every training step. Before landmark selection, `DiffusionMoELayer`'s own `cosine` flag (constructor param, default `False`) optionally L2-normalizes the activations first, so the kernel operates on cosine/angular distance instead of raw Euclidean — per-layer, via `DiffusionMoETransformer`'s `cosine_layers` (a set of layer indices, mirroring `layers_to_replace`'s pattern), not a project-wide switch: the full 32-layer multiscale sweep (findings report §2.4) found cosine isn't a uniform improvement, only a fix for the specific layers that never connect under raw Euclidean at any bandwidth (17, 20, 31 for Mistral-7B).
2. **Scale normalization** — `landmark_scale` (mean pairwise landmark-to-landmark distance, `routing/separation.py`) rescales `Psi_t` and the centroids before anything sees them. **This step exists only because of a real bug** (`phase1_findings_report.md` §3.4): raw diffusion coordinates measured $\approx 1e-7$ in magnitude on real Mistral-7B activations, several orders of magnitude too small for a fixed `tau=0.1` to produce anything but a numerically-uniform softmax — this rescaling is what makes `tau` operate in a consistent, dimensionless unit instead of each layer's own unpredictable raw coordinate scale.
3. **Routing** (`routing/router.py::DiffusionRouter`) — `router_logits = -||Psi_t - centroid_k||2` for every expert `k` (expanded via the `||a-b||2 = ||a||2 - 2a·b + ||b||2` identity, no explicit token-by-token loop), tempered softmax by `tau`, then top-`k` selection with gate values renormalized to sum to 1 over just the selected experts.
4. **Dispatch and aggregation** (`models/moe_dispatch.py::dispatch_and_aggregate`, shared by all 4 MoE variants) — tokens are gathered into contiguous per-expert buckets (`argsort` on flat expert assignment + `index_select`, so each expert runs once over its assigned tokens rather than once per token), each expert (`models/expert_ffn.py::ExpertFFN`, a SwiGLU sized to `ffn_dim // n_experts * overlap_factor`) processes its bucket, outputs are scattered back to original token order and weighted by gate value, and a token's (up to) `top_k` expert outputs are summed.
5. **Residual add** back onto the pre-FFN residual stream `x` (which already includes the attention residual from step 0) — standard pre-norm block structure, just with the FFN sub-layer replaced.

Expert centroids (`routing/centroids.py::ExpertCentroids`) are the one MoE-specific learned parameter beyond the experts themselves: `nn.Parameter` of shape `(n_experts, n_components)`, seeded via k-means++ on the *first* batch of diffusion coordinates the layer ever sees

(`initialise_from_batch` , a one-time no-op after that), then trained like any other parameter — pulled toward nearby tokens by the routing softmax's gradient, pushed apart by `centroid_separation_loss` (§6). `clip_norm_()` is a post-optimizer-step safeguard (not part of the loss itself) capping each centroid's norm at a multiple of the current `landmark_scale` , added after a real training run found the separation loss has no upper bound on centroid growth by design and let centroids drift far enough outside the data's real range to collapse the router toward uniform dispatch (`phase1_findings_report.md` §3.4, bug 1).

5. The four router variants (`_ROUTER_CLASSES` in `models/moe_model.py`)

All four share the identical attention → residual → norm → dispatch_and_aggregate → residual skeleton (§4's steps 0, 4, 5) and the identical `ExpertFFN` expert design; they differ *only* in how `router_logits` / `gate_values` / `expert_indices` are produced — deliberately, so any performance difference between them isolates the routing signal itself (project guide Section 11's baseline-comparison design):

Variant	Class	Routing signal	Learned routing params
diffusion (default)	<code>DiffusionMoELayer</code>	Negative squared distance in a diffusion-map embedding of z (§4)	<code>ExpertCentroids</code> (<code>n_components</code> -dim diffusion space)
cosine	<code>CosineMoELayer</code>	<code>cosine_similarity(z, centroid_k)</code> , centroids in raw <code>d_model</code> space	<code>nn.Parameter(n_experts, d_model)</code> centroids
switch	<code>SwitchMoELayer</code>	A single learned linear projection, softmax, top-k — no geometric structure at all	The linear router's weights (<code>Linear(d_model, n_experts)</code>)
random	<code>RandomMoELayer</code>	i.i.d. uniform random scores, top-k	None — no router parameters

What each baseline isolates: `diffusion` is the project's actual contribution, and the only variant using `NystromDiffusionMap` . `cosine` isolates whether diffusion-map geometry specifically helps versus any learned geometric notion of similarity in the raw residual stream. `switch` is the standard Switch Transformer (Fedus et al., 2021) baseline — it also computes the paper's own `switch_load_balance_loss` ($n_experts * \sum f_i \cdot P_i$) in its aux dict alongside the generic CV² loss, so both loss conventions are available for comparison. `random` isolates whether routing of *any* kind matters versus an unconditional mixture; its `router_logits` is fixed at all-zeros (uniform softmax — "balanced by construction") since there's nothing to log otherwise.

Note the naming collision to watch for: `CosineMoELayer` 's "cosine similarity" (its *routing metric*, operating on raw residual-stream vectors) is unrelated to `DiffusionMoELayer` / `PilotMoEBlock` 's own `cosine` flag (§4, stage 1) for the diffusion map's *kernel distance metric*. Both use the word "cosine" for genuinely different things at different stages of the pipeline — a `router="diffusion"` layer with `cosine=True` and a `router="cosine"` layer are unrelated settings that happen to share a name.

6. Training losses (`training/losses.py::total_loss`)

```
loss = task_loss + mu * load_loss + nu * sep_loss
```

- **task_loss** — standard causal-LM cross-entropy (`IGNORE_INDEX = -100` masking, matching `data/dataset.py::collate_fn` 's convention).
- **load_loss** (`routing/load_balance.py::coefficient_of_variation_loss`) — squared coefficient of variation of each expert's *dense* (pre-top-k, untempered softmax of `router_logits`) average gating mass across the batch. 0 for a perfectly balanced router; its maximum is exactly `n_experts - 1` for total collapse onto one expert — this ceiling is what the pilot fine-tune's oscillation (`phase1_findings_report.md` §3.4) was measured hitting exactly. Computed identically across **all four** router variants (every variant's aux dict carries dense `router_logits`), so load-balance is directly comparable between them.
- **sep_loss** (`routing/separation.py::centroid_separation_loss`) — negative mean pairwise distance between expert centroids in diffusion space, normalized by `landmark_scale` ; only computed for layers whose aux dict carries `centroids / Psi_landmarks` (currently `DiffusionMoELayer` only — `CosineMoELayer` 's raw-space centroids aren't put through this loss). Both `load_loss` and `sep_loss` are averaged across every MoE layer present, and are exactly 0 for a fully dense model (empty `router_outputs`).
- **mu** , **nu** — the two loss weights, `0.01 / 0.05` by default (`configs/base_config.yaml` 's `routing.mu_load / routing.nu_sep`). §7's tuning attempts (also `phase1_findings_report.md` §3.4) tested `mu` up to `0.3` without conclusively resolving load-balance oscillation at the one layer tested so far.

7. Configuration reference (`configs/`)

base_config.yaml — the routing/training hyperparameters actually used across this investigation's diagnostics and the pilot fine-tune (some pilot runs override individual values via CLI flags, noted where they do):

Key	Value	Notes
<code>routing.n_experts</code>	8	
<code>routing.top_k</code>	2	
<code>routing.tau</code>	0.1	Revised understanding, not value — see §4 stage 2 and <code>methodology.md</code> §2
<code>routing.n_components</code>	32	Diffusion embedding dimensionality
<code>routing.n_landmarks</code>	128	Nystrom landmark count
<code>routing.diffusion_t</code>	3	Diffusion-map low-pass sharpness
<code>routing.alpha</code>	1.0	Coifman–Lafon degree-normalization exponent
<code>routing.mu_load</code>	0.01	Load-balance loss weight
<code>routing.nu_sep</code>	0.05	Separation loss weight

routing.centroid_refresh_steps	500	Production default — the pilot used a much shorter 20 (and, in later tuning, 10,000)
routing.centroid_max_radius_factor	3.0	Caps centroid norm at this multiple of landmark_scale, applied by Trainer.train_step after every optimizer step
routing.noise_std	0.0 (off)	Noisy top-k gating magnitude — see §6; scale relative to tau, not absolute (findings report §3.5)
training.lr	3.0e-4	AdamW
training.total_tokens	30,000,000,000	Full Phase 2 training budget (not run yet)
training.precision	bf16	
layers_to_replace	null (= every layer)	See §2
cosine_layers	null (= none)	Per-layer cosine flag for the layers in layers_to_replace — see §4 stage 1

Two target model sizes (`configs/model/300m.yaml` , `1.3b.yaml`) describe the project's own from-scratch architectures for eventual Phase 2 training — **neither has been run in this investigation**; every diagnostic and the pilot fine-tune instead used the pretrained `mistralai/Mistral-7B-v0.1` as a stand-in dense reference (`methodology.md` §1 explains why):

	300M config	1.3B config
d_model	1024	2048
n_heads	16	16
n_layers	24	24
ffn_dim (dense-equivalent width)	4096	8192
max_seq_len	2048	2048

Both default to `router: "diffusion"` ; a dense baseline at either size is obtained via `layers_to_replace: []` , not a separate config.

8. Where the pilot fine-tune's architecture differs from the above

`models/pilot_moe_block.py::PilotMoEBlock` (used by `scripts/pilot_finetune.py` , `phase1_findings_report.md` §3.4) reuses every component in §4 (`NystromDiffusionMap` , `ExpertCentroids` , `DiffusionRouter` , `ExpertFFN` , `dispatch_and_aggregate`) completely unmodified — but is a narrower splice than a full `DiffusionMoELayer` : it replaces only a pretrained, frozen Mistral-7B decoder layer's `.mlp` submodule (`forward(x) -> Tensor` , matching HF's expected signature), not a whole from-scratch transformer block with its own attention. Auxiliary routing info is stashed on `self.last_aux` (read out as `{layer_idx: block.last_aux}`) rather than returned as a

tuple, since the surrounding frozen HF layer only expects a plain tensor back. Full detail in `methodology.md §4.2`.

9. What's validated vs. still assumed

This document describes the architecture as implemented; it makes no claim about which parts are empirically justified. For that, see `phase1_findings_report.md` :

- The core routing bet (diffusion-coordinate distance predicts useful expert specialization) has genuine, layer-dependent support (§3.3) and was shown trainable at one layer (§3.4) — but two real bugs in the exact code described in §4/§6 above were found only by training (now fixed; see `methodology.md §4.4`), and training stability at the default hyperparameters remains open.
- The choice of `router="diffusion"` over the `cosine / switch / random` baselines in §5 has not yet been tested empirically anywhere in this investigation — no run in this project has trained more than one router variant to compare.
- `layers_to_replace` 's selective-architecture use case (§2) is motivated by §3.3's 5-layer sample; the full 32-layer sweep needed to actually choose which layers is in progress at the time of writing.